# Project Report: Malignancy Detection in Cellular Biopsies

**Student Name:** [your name]
**Course / Batch, Institute:** [your course, batch and institute]
**Period of Internship:** 21st September 2026 – 16th October 2026
**Report submitted to:** IDEAS – Institute of Data Engineering, Analytics and Science Foundation, ISI Kolkata
**Project Mentor:** Dr. Chandan Biswas
**Draft date:** Oct 11, 2026

## Abstract

In this project I built models that predict whether a breast tumour is malignant or benign from measurements of cell nuclei. I used the Breast Cancer Wisconsin (Diagnostic) dataset that comes with scikit-learn. It has 569 samples and 30 features, with no missing values, and about 37% of the samples are malignant. I scaled the features, added one new feature (mean area divided by worst area) and split the data 80/20 with stratification. I removed outliers from the training set using Isolation Forest and then used PCA to bring the 31 features down to 10 components. My baseline, Logistic Regression, got about 98.6% accuracy in cross-validation and 98% on the test set. I also tuned a neural network (MLP) with grid search, and it gave almost the same ROC-AUC (0.996 compared to 0.997). Because missing a malignant case is much more serious than a false alarm, I also scored the models with a penalty of 10 for each false negative and 1 for each false positive. In the end, the simple model worked about as well as the neural network on this dataset.

## Introduction

Breast cancer is one of the most common cancers, and finding it early makes treatment more likely to work. In a biopsy, a pathologist looks at cells under a microscope and notes things like the size, shape and texture of the nuclei. Machine learning can learn patterns from these measurements and support the diagnosis.

I used the Breast Cancer Wisconsin (Diagnostic) dataset from `sklearn.datasets`. It has 569 samples and 30 features. Ten properties of the nuclei (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension)

are given as a mean, a standard error and a "worst" (largest) value. The target is 0 for malignant and 1 for benign.

I worked in Python with pandas, NumPy, matplotlib, seaborn and scikit-learn, in Google Colab. My goal was to go through the whole workflow once: checking the data, preparing it, building models, tuning them and judging the results. I kept in mind that in medicine a missed malignant case is much worse than a false alarm.

**Topics covered in the first two weeks of the internship:** [list the topics from your training here]

## Project Objective

1. Load and inspect the dataset, and check for missing values and class imbalance.
2. Find highly correlated features and explain why they matter for linear models.
3. Prepare the data: scaling, one engineered feature, a stratified 80/20 split, outlier removal and PCA.
4. Build and compare a Logistic Regression baseline and a tuned MLP using cross-validation, a classification report and ROC-AUC.
5. Judge the models with a custom penalty that treats a missed malignant case as 10 times worse than a false alarm, and find the features that matter most.

## Methodology

No survey was done. The data comes from the built-in scikit-learn dataset, so no separate data collection was needed. The steps were:

1. **Loading:** I put the 30 features into a pandas DataFrame and added the diagnosis as a `target` column.
2. **Checking the data:** I counted the missing values in each column and made a bar chart of the two classes.
3. **Correlation:** I drew a heatmap of the 30 features. Using only the upper triangle so each pair shows once, I listed the pairs with a correlation above 0.90 (absolute value).
4. **Scaling:** I used `StandardScaler` so every feature has mean 0 and standard deviation 1, and turned the result back into a DataFrame to keep the column names.
5. **New feature:** I calculated mean area divided by worst area from the original values, and then standardised it like the others.

6. **Split:** 80% for training (455 rows) and 20% for testing (114 rows), stratified on the target, with `random_state=42`.
7. **Outliers:** I fitted `IsolationForest` (contamination 0.05) on the training features only and removed the rows it flagged from the training features and labels. I did not touch the test set.
8. **Baseline model:** I checked Logistic Regression with 5-fold stratified cross-validation, then trained it on the whole cleaned training set and tested it.
9. **PCA:** I fitted PCA on the training data only, kept enough components for 95% of the variance, and applied it to the test data.
10. **MLP and tuning:** I tuned an `MLPClassifier` (ReLU, `max_iter=500`) with `GridSearchCV` (3-fold) on the PCA training data, trying three hidden-layer layouts, (32, 16), (64, 32) and (16,), and two learning rates, 0.001 and 0.01.
11. **Comparison:** I compared both models with ROC-AUC (malignant as the positive class), scored the MLP with the custom penalty, and used the Logistic Regression coefficients to find the most important features.

**Tools:** Python, pandas, NumPy, matplotlib, seaborn, scikit-learn, Google Colab.

**Code:** [paste your GitHub link here]

[Optional: insert a simple flow chart of the steps above]

# Data Analysis and Results

## Descriptive analysis

| Item | Result |
|---|---|
| Samples / features | 569 / 30 (plus the target) |
| Missing values | 0 |
| Malignant / benign | 212 (37.3%) / 357 (62.7%) |
| Feature pairs with absolute correlation above 0.90 | 21 (highest: mean radius and mean perimeter, 0.998) |
| Train / test split | 455 / 114 rows, malignant share 37.4% / 36.8% |
| Outliers removed from training | 23 (432 rows left) |

| Item | Result |
|---|---|
| PCA components for 95% variance | 10 (95.5% retained) |

[Insert screenshots: class distribution bar chart, correlation heatmap, confusion matrix, ROC curves]

The classes are a little imbalanced, so I did not rebalance them. I used a stratified split and looked at recall, F1 and ROC-AUC along with accuracy. A lot of the size features (radius, perimeter, area) measure almost the same thing, which is why so many pairs are highly correlated.

## Model comparison

| Measure | Logistic Regression | Tuned MLP (on PCA data) |
|---|---|---|
| Cross-validation accuracy | 0.9861 (5-fold, std 0.0046) | 0.9838 (3-fold, grid search best) |
| Test false negatives | 1 | 1 |
| Test false positives | 1 | 2 |
| ROC-AUC on test set | 0.9974 | 0.9957 |
| Diagnostic penalty (10 per FN, 1 per FP) | 11 | 12 |

The best MLP used hidden layers (32, 16) and a learning rate of 0.001. For Logistic Regression, the test report showed precision, recall and F1 of about 0.98 for malignant and 0.99 for benign, with 98% accuracy overall.

## Most important features

The three biggest Logistic Regression coefficients, ignoring the sign, were the mean-to-worst area ratio I created (+1.359), worst texture (-0.975) and mean concave points (-0.941). The model predicts benign as class 1, so a negative coefficient means that a larger value points towards malignant.

No hypothesis testing or survey was part of this project.

# Conclusion

Both models separated malignant from benign cells very well. Logistic Regression got 98% test accuracy, a ROC-AUC of 0.997 and a penalty of 11. The tuned MLP got a ROC-AUC of 0.996 and a penalty of 12. The difference is very small with only 114 test samples, so the neural network and PCA did not really help here. The simpler model is also easier to explain, which matters in medicine.

There are some limits. The dataset is small, I used a single train/test split, and all the data comes from one source. I also chose the outlier rate (5%) and the new feature by hand, without tuning them.

In the future I could try other models like Random Forest or SVM, adjust the decision threshold to reduce false negatives, test on a different dataset, and use the penalty score while tuning instead of accuracy.

# Appendices

### A. References

- scikit-learn documentation: scikit-learn.org
- Breast Cancer Wisconsin (Diagnostic) dataset, as provided in `sklearn.datasets.load_breast_cancer`
- [add any other tutorials, papers or course material you used]

**B. Other documents (notebook, video, report copy):**
https://colab.research.google.com/drive/1Ch3Bcc1HUH3axoT8EK_yw6No2bdMeHh5#scrollTo=q15-code